Git and GitHub

Version control System (VCS):

Tool or software that helps you to track changes made to files over time. It lets you:

- Save different versions of projects
- Go back to an earlier version
- Work with team
- See who made change what and when

Examples of Version Control System (VCS)?

Examples of VCS:

NAME	TYPE	DESCRIPTION
Git	Distributed VCS	Most popular. Used by developers to track code changes.
Subversion (SVN)	Centralized VCS	Older system, all files stored in a central server.
Mercurial	Distributed VCS	Similar to Git, used for performance and simplicity.
CVS	Centralized VCS	One of the first version control systems, now outdated.
Perforce	Centralized VCS	Used in large enterprises and gaming companies.

•**Centralized VCS** – All data is stored in one place (e.g., SVN).
•**Distributed VCS** – Every user has a full copy (e.g., Git, Mercurial).

What is a GitHub?

GitHub is a website where you can store your **Git Projects** online.

Launched by:
Tom Preston-Werner, Chris Wanstrath, and PJ Hyett in 2008.

Owned by:
Microsoft Since 2018.

- Online hosting of Git repositories (Foldes) Repo
- Team collaboration and code sharing
- Issue tracking for bugs and tasks
- Public and private repositories

What is a Git?

Its a **version control system** used to track changes in code or files.

Created by: Linus Torvalds in 2005.

- Free and open-source
- Distributed version control system
- Fast ,Secure and reliable
- Tracks changes with full history
- Enables team collaboration

What is a Git Bash?

A **command-line tool** (terminal) that allows you to run **Git commands**. Comes bundled when you install Git on Windows.

What is GitHub Repository:

Git Add & Commit

In Git, add and commit are two basic steps to save changes in your project.

Step 01 → add — add tells Git which files you want to include in your next save.

Step 02 → commit — commit saves those changes with a message, so you have a record of what you did.

Git Configuration Commands

To configure git into your system or to link your local computer with github, you need to use following git commands.

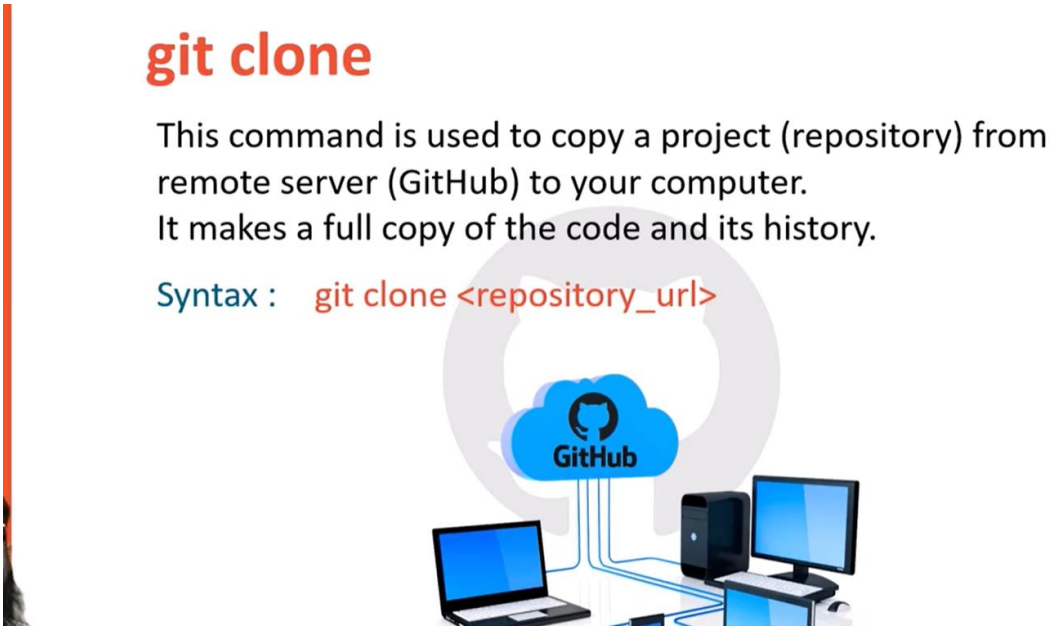
```
git config - - global user.name “github-username”

git config - - global user.email “github-email”

git config - - list
```

git config --global user.name "bilalramzan01"

We'll walk through: git config --global user.name "Your Name"

git config --global user.email "bilalramzan.cs@gmail.com"

git config --global --list



Terminal Commands

cd (Change Directory)

This command is used to move from one folder (directory) to another in your computer through the terminal or command line.

Syntax : cd <folder_name>

Examples:

- cd Documents (To move inside of Documents Folder)
- cd .. (To go back or move to previous folder)
- cd /home/user/Desktop
  (This directly takes you to the Desktop folder.)

Terminal Commands

ls (List)

- Shows a list of files and folders in the current directory (folder).

clear

- Clears the terminal screen to remove all previous commands.

pwd (Print Working Directory)

- Shows the full path of the current folder you are in.

git clone

Purpose: Download an existing repository from GitHub to your computer.

git clone https://github.com/user/repo.git

git status

Purpose: Check the current state of your repository.

git status

📌 Shows:

- modified files
- staged files
- untracked files

git add

Purpose: Add files to the staging area before committing.

git add file.txt

git commit

Purpose: Save changes permanently in Git.

git commit -m "Added login page"

📌 Always write a meaningful message.

git push

Purpose: Upload local commits to GitHub.

git push origin main

📌 Sends your code to the remote repository.

git pull

Purpose: Download the latest changes from GitHub.

git pull origin main

📌 Updates your local repository.

git branch

Purpose: Show or create branches.

Show branches:

git branch

Create new branch:

git branch feature-login

git checkout

Purpose: Switch between branches.

git checkout feature-login

git merge

Purpose: Combine branches.

git merge feature-login

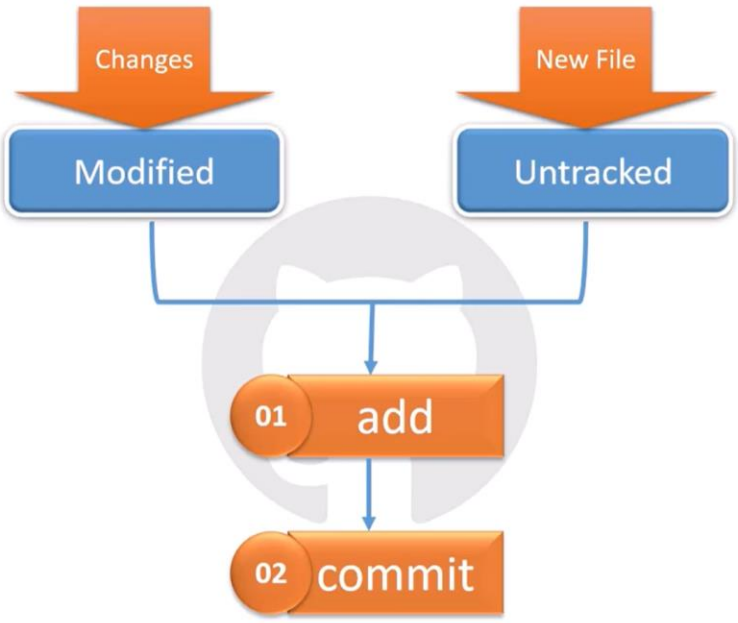
git status

1. Nothing to commit, working tree clean
Everything is up to date. No changes to commit.

2. Modified
Message: Changes not staged for commit:
The content of the file has changed.

3. Untracked files
You created a new file, but haven't added.

4. Staged
Message: Changes to be committed:
A file is ready to be committed.



add & commit

git add

Adds your changed or new files to the ***Staging Area*** (prepares files for commit).

Syntax: git add <filename.txt>

(OR)

To add all files: git add .

add & commit

git add

Adds your changed or new files to the ***Staging Area*** (prepares files for commit).

Syntax: git add <filename.txt>

(OR)

To add all files: git add .

git commit

Saves the staged changes to your local Git repository & keep its record.

Syntax: git commit -m "Write your message here"

Git Push Command

git push

This command sends (uploads) your committed changes from your local computer to a remote repository like GitHub.

Example: git push origin main

git init Command

This command is used to **start a new Git Repository** in your project folder.

Syntax git init [directory]

Explanation:

- git init ➔ initializes a Git repository in the current folder.
- git init my-folder ➔ initializes a Git repository in the specified folder (creates the folder if it doesn't exist)

Push Repo on GitHub

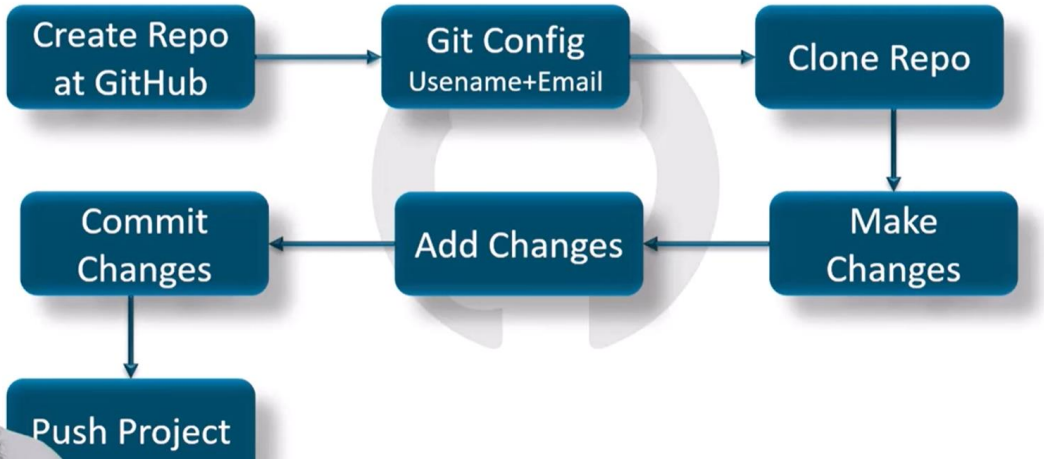
Follow these steps to push this repo on GitHub.

01 Make a new repo at GitHub with same name. (No need to initialize readme file)

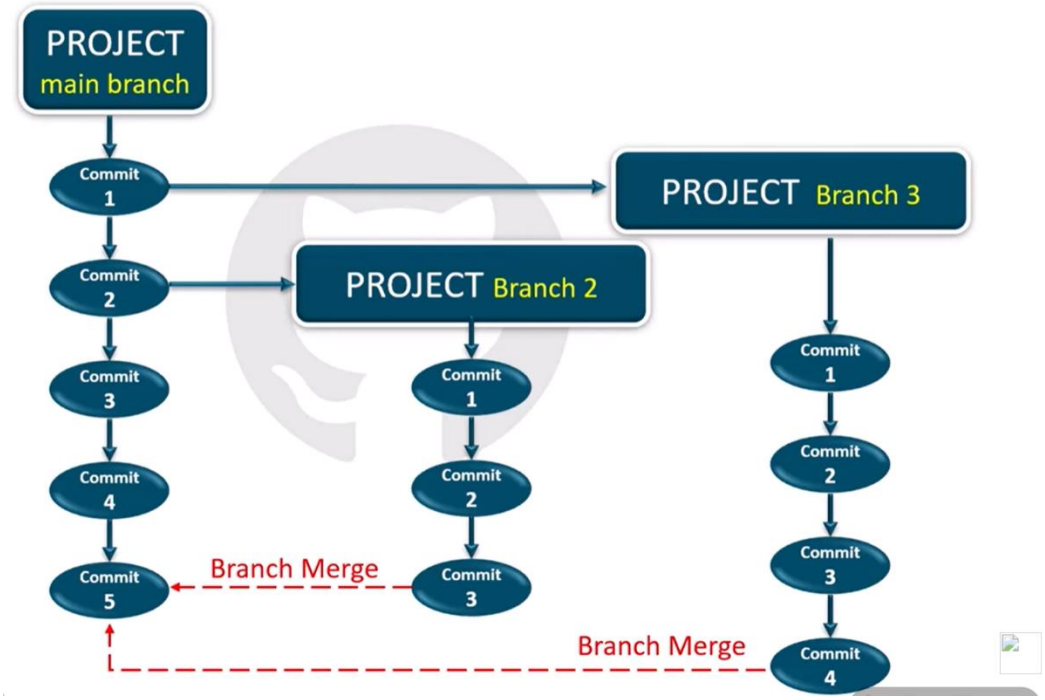
02 git remote add origin <link>
Send my project to this online link

- git remote add — means you are adding a remote connection.
- origin — is the name you give to the remote (default is "origin").
- <link> — is the URL of your GitHub (or other) repo.

Work Flow



Git Branches



Git Branch Commands

git branch

See all branches (OR) create a new one.

- Example:
  - git branch → Shows all branches.
  - git branch new-branch → Creates a new branch.

git branch -m

Rename a branch.

- Example:
  - git branch -m old-name new-name

Git Branch Commands

git checkout

Switch to another branch.

- Example:
  git checkout main → Switches to the main branch.

git checkout -b

Create and switch to a new branch.

- Example:
  git checkout -b function1 → Creates & moves to function1 branch.

Git Branch Commands

git branch -d

Delete a branch (if already merged).

- Example:
  git branch -d function1 → Deletes function1 branch.

git branch -D

Force delete a branch (even if not merged).

- Example:
  git branch -D function1 → Force deletes function1.

Git Branch Commands

git diff

Used to compare commits,files, branches etc.

- Example:

  git diff function1→ compare function1 branch with current branch.

Merge Code

git pull

git pull is a Git command used to download the latest changes from a remote repository (GitHub) and add them to your local project.

Example:

git pull origin main

Merge Conflict

What is a Merge Conflict?

A merge conflict happens when Git cannot automatically merge changes from different branches because the same lines in the same file were changed in different ways.

Method : 01

git merge

Combine changes from one branch into another.

Method : 02

Create PR (Pull Request) using GitHub

Undo Staged Change

(Changes add but not commit)

git reset is a Git command used to undo changes.

git reset <filename>

git reset

Undo committed Change
(Changes add and commit)

git reset HEAD~1

git reset --hard HEAD~1 Delete the last commit and all changes:

Undo commited Changes
(Jump to Specific commit)

git reset <commit id>

git reset --hard <commit id>

```
PS C:\DummyProject\LocalProject> git log --oneline
7d5493e (HEAD -> main) new line added
2db8f08 Changes by Tariq Mahoomd
2df18a3 (origin/function1) Changes made in html
b0ba3ec Readme file created
8db439a CSS File Created
3d0ad17 New HTML file created
PS C:\DummyProject\LocalProject>
```

Fork

A **fork** is a personal copy of someone else's repository that is created under your own GitHub account. It allows you to freely experiment with changes without affecting the original project.

Key Points about Fork:

- It creates a **separate copy** of the repository.
- You can **edit, modify, or add new features** in your forked repo.
- Commonly used to contribute to **open-source projects**.
- After making changes, you can send a **pull request** to the original repo owner to suggest merging your changes.